

Phase 1 Post-Mortem

Consolidating Every Event Mutation Through One Seam

Date: 2026-09-14

Commit: 4d5ef44 on main

Scope: Internal architecture only. Zero user-facing UI change. Zero runtime behaviour change.

The one-paragraph version

Every place in the codebase that used to write to an event object directly (like `event.outcome = 'neutralized'` or `event.linkedEventIds.push(...)`) now goes through a small set of functions exported from `events.js`. A build-time check refuses to compile if any code tries to sneak a direct write in. The platform behaves identically. The architecture just got noticeably cleaner.

Why this mattered

Before Phase 1, event state was a shared mutable pond that seven different files were writing to. `events.js` owned the array but any file could reach in and change any field. There were **79 direct writes across 7 files** — three of them for the Post-Incident Report alone, on three different code paths.

Practical consequence: adding a new field to an event required a manual audit of every possible write site to figure out what the new field would interact with. In six months, that audit would have gotten expensive. In twelve, painful.

Second practical consequence: when Azure lands and we need to persist events server-side, the "hook point" for that swap was 79 different lines of code. Now it's a handful of functions in one file.

Before vs After — one concrete example

Before — inside `main.js` when a fighter jet neutralised a hostile target:

```
event.outcome = 'neutralized';
event.neutralizedAt = new Date().toISOString();
event.neutralizedBy = 'Flyvevåbnet Fighter Response, Skrydstrup';
event.projectedPath = null;
event.needsPostIncident = true;
```

Five separate writes. Five different fields. Nothing tying them together as one lifecycle event. If the future Azure sync needs to fire on every neutralisation, you'd have to find every place in the code that writes `event.outcome = 'neutralized'` and add the sync there.

After — same behaviour, one call:

```
markNeutralised(event.id, {
  outcome: 'neutralized',
  by: 'Flyvevåbnet Fighter Response, Skrydstrup',
  needsPostIncident: true,
});
```

The fields still get written the same way. But now there's a single function in `events.js` called `markNeutralised` that owns the whole lifecycle transition. Add the Azure sync there once, every neutralisation path gets it automatically.

What the new API looks like

Two tiers.

Tier 1 — Primitives (silent, low-level)

These are for miscellaneous field writes. They don't automatically trigger UI re-renders (because many of them fire 60 times per second inside the tick loop and would cause a render storm).

Function	Use for
<code>mutateEvent(id, patch)</code>	Setting one or more scalar/object fields
<code>appendEventArray(id, field, item)</code>	Pushing to an array field
<code>addToEventSet(id, field, item)</code>	Adding to a Set field
<code>setEventMapKey(id, field, key, value)</code>	Setting a key on a Map field

Tier 2 — Semantic (fire listeners, meaningful transitions)

These represent meaningful lifecycle events. They batch related fields together and fire UI listeners so the interface updates.

Function	Represents
<code>linkEvents(idA, idB, score)</code>	Two events are the same drone — link bidirectionally
<code>markNeutralised(id, {...})</code>	Target is down — stamp outcome, timestamp, actor, cleanup
<code>recordInteraction(id, item)</code>	Adapter fired — log to interactions array
<code>updateEventSubject(id, subject)</code>	NN classification refreshed — attach + clear narrative cache
<code>attachPostIncidentReport(id, pir)</code>	PIR generated — stamp on event
<code>clearNarrativeCache(id) / setNarrativeCache(id, x)</code>	Narrative cache lifecycle
<code>clearPreprocessedCache(id) / setPreprocessedCache(id, x)</code>	Preprocessing cache lifecycle

The safety net

`scripts/check-event-mutations.mjs` runs a grep over the codebase looking for any line that matches `event.something = ...` or `event.something.push(...)` etc., outside of `events.js`.

It's wired into `npm run build` — the build refuses to compile if a violation is found. Try it:

```
$ npm run check:event-mutations
✓ Event mutation policy satisfied. Every event write goes through events.js.
```

If someone later adds `event.foo = bar` in a random file, the build fails with a pointer to the exact line and file, plus a note directing them to the mutator API. That's the entire safety mechanism. No runtime overhead. No proxy tricks. Just a grep that runs before Vite builds.

One documented exemption: `detection_subject.js` contains `event.subject = next` on one line. It's exempt because that function is only ever called from `events.js` itself — it's an internal helper, not an external caller. The exemption is annotated inline with a comment so the safety net skips that specific line.

The tick-loop performance concern (and how it was handled)

The tick loop runs at 60 frames per second. During a hostile-drone simulation, it might be updating 10-30 events per tick, with 5-15 fields written per event. That's up to 27,000 field writes per second.

If every one of those writes triggered a UI re-render (fired the `_listeners` fanout), the interface would freeze.

Solution: primitives are silent by default. They write the field, don't fire listeners. The tick loop already orchestrates its own render at the end of each tick, so the UI still updates correctly.

Semantic mutators (the meaningful lifecycle transitions above) DO fire listeners, because those represent moments where the UI genuinely needs to react — a neutralisation, a new linked event, an adapter interaction.

Callers who want a listener fire on a primitive can opt in explicitly: `mutateEvent(id, patch, { notify: true })`.

What this unlocks (immediately)

For iteration speed

Adding a new field to an event is now: define it in the mutator that owns its lifecycle, done. No more grep-hunting through `main.js` for `event.oldField =` to figure out where the new field should also be set.

For Phase 3 (tenant isolation)

The tenant model needs every event to carry a `tenantId` stamped at creation time, and every filter to intersect it against the actor's tenant. Because mutations now flow through mutator functions, adding the `tenantId` field is one line in `addEvent`. No sweep of 79 callsites required.

For Azure persistence (when it lands)

The Azure event-persistence swap will hook every mutator function to also write through to a server-side store. One file (`events.js`), 14 functions, done. Before this refactor, the same swap would have required editing 79 lines across 7 files, plus adding a wrapper around every direct assignment.

What this does NOT do

Being honest about the scope. Phase 1 was strictly about the write seam.

- **Rendering is not extracted from `main.js`.** That's Phase 2. `main.js` is still 22,600 lines with template strings inline.
 - **Tenant isolation is still UI-level.** That's Phase 3. Events are still a single global array with client-side filtering.
 - **Nested writes (like `event.dispatchOutcomes[key] = value` or `event.subject.class = ...`) are not caught by the safety net.** The current scope is top-level field writes only. Nested writes are a Phase 1.5 follow-up if they start causing issues.
 - **No new UI features. No new architectural surface. Nothing visible in the browser.**
-

The verdict

Metric	Before	After
Files that mutate event objects	7	1 (<code>events.js</code>)
Files that mutate event objects (audit exempt)	0	1 (<code>detection_subject.js</code> , called only from <code>events.js</code>)
Direct write callsites	79	0

Mutator API functions	8 existing	8 existing + 14 new = 22
Build-time safety check	none	grep-based, refuses to compile on violation
Runtime perf impact	baseline	zero measurable — primitives are silent, tick loop unchanged
User-facing behaviour	baseline	identical
Lines added	—	422
Lines removed	—	134

Phase 1 done. Phase 2 (rendering carve-out from main.js) and Phase 3 (tenant isolation at data layer) are up next, both unblocked by this landing.